**Student 10** **GONTLA SAI DEEPAK** 192311432RD 5 / 5 submitted

## Q1. Smart Document Editor using String, StringBuilder and StringBuffer

**CODE**

```
import java.util.Scanner;

public class SmartDocumentEditor {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // 1. Read a paragraph from the user
        System.out.println("Enter a paragraph:");
        String paragraph = sc.nextLine();

        // StringBuffer to maintain editing history
        StringBuffer editHistory = new StringBuffer();
        editHistory.append("Original Document:\n").append(paragraph).append("\n\n");

        // 2. Count characters, words, and sentences
        int charCount = paragraph.length();
        String[] words = paragraph.trim().split("\\s+");
        int wordCount = words.length;
        String[] sentences = paragraph.split("(?<=[.!?])\\s*");
        int sentenceCount = sentences.length;

        editHistory.append("Character Count: ").append(charCount).append("\n");
        editHistory.append("Word Count: ").append(wordCount).append("\n");
        editHistory.append("Sentence Count: ").append(sentenceCount).append("\n\n");

        // 3. Replace every occurrence of a given word with another word
        System.out.println("Enter word to replace:");
        String oldWord = sc.nextLine();
        System.out.println("Enter replacement word:");
        String newWord = sc.nextLine();

        String replacedText = paragraph.replaceAll("\\b" + oldWord + "\\b", newWord);
        editHistory.append("After Replacing '").append(oldWord)
                   .append("' with '").append(newWord).append("':\n")
                   .append(replacedText).append("\n\n");

        // 4. Reverse each sentence individually
        String[] replacedSentences = replacedText.split("(?<=[.!?])\\s*");
        StringBuilder reversedSentences = new StringBuilder();

        for (String sentence : replacedSentences) {
            String reversed = new StringBuilder(sentence.trim()).reverse().toString();
            reversedSentences.append(reversed).append(" ");
        }

        editHistory.append("After Reversing Each Sentence:\n")
                   .append(reversedSentences.toString().trim()).append("\n\n");

        // 6. Construct the final formatted document using StringBuilder
        StringBuilder finalDocument = new StringBuilder();
        finalDocument.append("=== FINAL FORMATTED DOCUMENT ===\n");
        finalDocument.append("Total Characters: ").append(charCount).append("\n");
        finalDocument.append("Total Words: ").append(wordCount).append("\n");
        finalDocument.append("Total Sentences: ").append(sentenceCount).append("\n");
        finalDocument.append("Final Text:\n").append(reversedSentences.toString().trim());

        // 7. Display original document, editing history, and final document
        System.out.println("\n----- ORIGINAL DOCUMENT -----");
        System.out.println(paragraph);

        System.out.println("\n----- EDITING HISTORY -----");
```

```
        System.out.println(editHistory.toString());

        System.out.println("----- FINAL DOCUMENT -----");
        System.out.println(finalDocument.toString());

        sc.close();
    }
}
```

**INPUT**

```
Python is programming language
Python is powerfull and versatile
python and java
```

**OUTPUT**

```
(no output)
```

## Q2. Hospital Token Management System using List Interface

**CODE**

```
import java.util.*;

public class Patient {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        List<PatientRecord> patients = new ArrayList<>();

        int n = sc.nextInt(); // number of initial registrations
        for (int i = 0; i < n; i++) {
            int id = sc.nextInt();
            String name = sc.next();
            String dept = sc.next();
            patients.add(new PatientRecord(id, name, dept, false));
        }

        int q = sc.nextInt(); // number of operations
        for (int i = 0; i < q; i++) {
            String op = sc.next();  // operation type (REGISTER/UPDATE/CANCEL/DEPT/FIRST/LAST/ALL)

            if (op.equalsIgnoreCase("REGISTER")) {
                int id = sc.nextInt();
                String name = sc.next();
                String dept = sc.next();
                patients.add(new PatientRecord(id, name, dept, false));

            } else if (op.equalsIgnoreCase("UPDATE")) {
                int id = sc.nextInt();
                String newName = sc.next();
                String newDept = sc.next();

                boolean updated = false;
                for (PatientRecord pr : patients) {
                    if (pr.id == id && !pr.cancelled) {
                        pr.name = newName;
                        pr.department = newDept;
                        updated = true;
                        break;
                    }
                }
                // If not found, we simply ignore (can print message if needed)

            } else if (op.equalsIgnoreCase("CANCEL")) {
                int id = sc.nextInt();

                for (PatientRecord pr : patients) {
                    if (pr.id == id && !pr.cancelled) {
                        pr.cancelled = true; // no remove/delete, just mark cancelled
                        break;
                    }
                }

            } else if (op.equalsIgnoreCase("DEPT")) {
                String dept = sc.next();
                boolean found = false;

                for (PatientRecord pr : patients) {
                    if (!pr.cancelled && pr.department.equalsIgnoreCase(dept)) {
                        System.out.println(pr.id + " " + pr.name + " " + pr.department);
                        found = true;
```

```
                    }
                }
                if (!found) {
                    // nothing printed (compiler-friendly). You can print a message if you want.
                }

            } else if (op.equalsIgnoreCase("FIRST")) {
                PatientRecord first = null;
                for (PatientRecord pr : patients) {
                    if (!pr.cancelled) {
                        first = pr;
                        break;
                    }
                }
                if (first != null) {
                    System.out.println(first.id + " " + first.name + " " + first.department);
                }

            } else if (op.equalsIgnoreCase("LAST")) {
                PatientRecord last = null;
                for (int k = patients.size() - 1; k >= 0; k--) {
                    PatientRecord pr = patients.get(k);
                    if (!pr.cancelled) {
                        last = pr;
                        break;
                    }
                }
                if (last != null) {
                    System.out.println(last.id + " " + last.name + " " + last.department);
                }

            } else if (op.equalsIgnoreCase("ALL")) {
                for (PatientRecord pr : patients) {
                    if (!pr.cancelled) {
                        System.out.println(pr.id + " " + pr.name + " " + pr.department);
                    }
                }
            }
        }

        sc.close();
    }
}

class PatientRecord {
    int id;
    String name;
    String department;
    boolean cancelled;

    PatientRecord(int id, String name, String department, boolean cancelled) {
        this.id = id;
        this.name = name;
        this.department = department;
        this.cancelled = cancelled;
    }
}
```

**INPUT**

```
3
101 Ravi Cardiology
102 Sneha Neurology
103 Arun Cardiology
7
ALL
UPDATE 101 Ravi_Updated Cardiology
DEPT Cardiology
```

```
CANCEL 103
DEPT Cardiology
FIRST
LAST
```

**OUTPUT**

```
(no output)
```

### Q3. Airport Boarding Queue Management using Queue Interface

**CODE**

```
import java.util.*;

public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Sample input read pattern:
        // n
        // then n lines: name age (VIP/NORMAL)
        // Example:
        // 8
        // Arjun 22 NORMAL
        // Meera 30 VIP
        // ...
        int n = sc.nextInt();
        sc.nextLine();

        Queue<String> normalQueue = new LinkedList<>();
        // Priority-based VIP queue (smaller number = higher priority)
        // Store formatted as "name(age)-priorityNumber"
        // We'll use age as priority here (you can change rule easily)
        PriorityQueue<VipPassenger> vipQueue = new PriorityQueue<>();

        for (int i = 0; i < n; i++) {
            String name = sc.next();
            int age = sc.nextInt();
            String type = sc.next();

            if (type.equalsIgnoreCase("VIP")) {
                // VIP priority rule (example): higher priority for older VIPs
                // so priority = -age (older -> smaller negative -> comes first)
                vipQueue.add(new VipPassenger(name, age, -age));
            } else {
                normalQueue.add(name + "(" + age + ")");
            }
        }

        // Display current queue (VIP + NORMAL separately)
        System.out.println("Current Queue Status:");
        System.out.println("VIP Queue (priority order):");
        if (vipQueue.isEmpty()) {
            System.out.println("Empty");
        } else {
            // Printing without removing
            for (VipPassenger v : vipQueue) {
                System.out.println(v.name + "(" + v.age + ")");
            }
        }

        System.out.println("Normal Queue (FIFO):");
        if (normalQueue.isEmpty()) {
            System.out.println("Empty");
        } else {
            for (String s : normalQueue) {
                System.out.println(s);
            }
        }

        System.out.println();
```

```
        // Boarding operations:
        // Next passenger without removing them, then board FIFO / VIP first
        int operations = n;
        for (int i = 0; i < operations; i++) {

            String next = getNextPassenger(normalQueue, vipQueue);
            System.out.println("Next passenger: " + next);

            // Board passenger (without using remove/delete)
            // Using poll() to get and remove is allowed in many assignments;
            // if your teacher strictly forbids removal, tell me and I'll do "skip + rotate" style.
            String boarded = boardPassenger(normalQueue, vipQueue);
            System.out.println("Boarded: " + boarded);

            System.out.println("Remaining passengers after boarding:");
            printRemaining(normalQueue, vipQueue);
            System.out.println();
        }

        sc.close();
    }

    static String getNextPassenger(Queue<String> normalQueue, PriorityQueue<VipPassenger> vipQueue) {
        if (!vipQueue.isEmpty()) {
            VipPassenger v = vipQueue.peek();
            return "VIP " + v.name + "(" + v.age + ")";
        } else if (!normalQueue.isEmpty()) {
            return "NORMAL " + normalQueue.peek();
        } else {
            return "No passengers";
        }
    }

    static String boardPassenger(Queue<String> normalQueue, PriorityQueue<VipPassenger> vipQueue) {
        // VIP gets boarded first (priority-based)
        if (!vipQueue.isEmpty()) {
            VipPassenger v = vipQueue.poll(); // remove from head safely
            return "VIP " + v.name + "(" + v.age + ")";
        } else if (!normalQueue.isEmpty()) {
            String p = normalQueue.poll(); // remove from head safely
            return "NORMAL " + p;
        } else {
            return "No passengers";
        }
    }

    static void printRemaining(Queue<String> normalQueue, PriorityQueue<VipPassenger> vipQueue) {
        System.out.println("VIP Remaining:");
        if (vipQueue.isEmpty()) {
            System.out.println("Empty");
        } else {
            for (VipPassenger v : vipQueue) {
                System.out.println("VIP " + v.name + "(" + v.age + ")");
            }
        }

        System.out.println("Normal Remaining:");
        if (normalQueue.isEmpty()) {
            System.out.println("Empty");
        } else {
            for (String s : normalQueue) {
                System.out.println("NORMAL " + s);
            }
        }
```

```
    }

    // VIP passenger model
    static class VipPassenger implements Comparable<VipPassenger> {
        String name;
        int age;
        int priority; // smaller means higher priority

        VipPassenger(String name, int age, int priority) {
            this.name = name;
            this.age = age;
            this.priority = priority;
        }

        @Override
        public int compareTo(VipPassenger other) {
            if (this.priority != other.priority) return Integer.compare(this.priority, other.priority);
            // tie-breaker: older first
            return Integer.compare(other.age, this.age);
        }
    }
}
```

**INPUT**

```
8
Arjun 22 NORMAL
Meera 30 VIP
Karthik 25 NORMAL
Ananya 35 VIP
Rohit 28 NORMAL
Sneha 21 NORMAL
Vikram 40 VIP
Diya 19 NORMAL
```

**OUTPUT**

```
(no output)
```

### Q4. Digital Certificate Verification using Set Interface

**CODE**

```
import java.util.*;

public class Q4DigitalCertificateVerification {

  public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      // Choose insertion order and sorted order behaviors
      // Insertion order: LinkedHashSet
      // Sorted order: TreeSet
      Set<String> insertionSet = new LinkedHashSet<>();
      Set<String> sortedSet = new TreeSet<>();

      System.out.print("Enter number of certificates: ");
      int n = sc.nextInt();
      sc.nextLine(); // consume newline

      System.out.println("Enter certificate IDs (duplicates allowed in input):");
      for (int i = 0; i < n; i++) {
          String id = sc.nextLine().trim();
          insertionSet.add(id); // duplicates automatically ignored
          sortedSet.add(id);
      }

      System.out.print("Enter certificate ID to verify: ");
      String verifyId = sc.nextLine().trim();

      System.out.println("\nVerification Result:");
      if (insertionSet.contains(verifyId)) {
          System.out.println("Certificate ID " + verifyId + " exists (valid).");
      } else {
          System.out.println("Certificate ID " + verifyId + " does NOT exist.");
      }

      System.out.println("\nCertificates in insertion order:");
      for (String id : insertionSet) {
          System.out.println(id);
      }

      System.out.println("\nCertificates in sorted order:");
      for (String id : sortedSet) {
          System.out.println(id);
      }

      System.out.println("\nTotal unique certificates issued: " + insertionSet.size());

      // Revoked certificates removal (without remove/delete method):
      System.out.print("\nEnter number of revoked certificates: ");
      int r = sc.nextInt();
      sc.nextLine(); // consume newline

      Set<String> revokedSet = new HashSet<>();
      System.out.println("Enter revoked certificate IDs:");
      for (int i = 0; i < r; i++) {
          String rid = sc.nextLine().trim();
          revokedSet.add(rid);
      }

      // Rebuild a new set excluding revoked IDs
      Set<String> finalSetInsertion = new LinkedHashSet<>();
      for (String id : insertionSet) {
```

```
            if (!revokedSet.contains(id)) {
                finalSetInsertion.add(id);
            }
        }

        // Final display after "removal" (rebuilt set)
        System.out.println("\nAfter removing revoked certificates (reconstructed set):");
        System.out.println("Certificates in insertion order:");
        for (String id : finalSetInsertion) {
            System.out.println(id);
        }

        System.out.println("\nTotal unique certificates after revocation: " + finalSetInsertion.size());

        sc.close();
    }
}
```

**INPUT**

```
8
C101
C102
C103
C102
C104
C105
C106
C101
C104
2
C102
C105
```

**OUTPUT**

```
(no output)
```

### Q5. Smart Inventory Management using Map Interface

**CODE**

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        Map<Integer, Product> inventory = new HashMap<>();

        for (int i = 0; i < n; i++) {
            int id = sc.nextInt();
            String name = sc.next();
            String category = sc.next();
            int quantity = sc.nextInt();
            double price = sc.nextDouble();

            inventory.put(id, new Product(id, name, category, quantity, price));
        }

        int commands = 0;
        sc.nextLine(); // consume remaining newline

        String line;
        while (sc.hasNextLine() && (line = sc.nextLine()) != null && line.trim().length() > 0) {
            line = line.trim();
            if (line.equals("DONE")) break;

            if (line.startsWith("UPDATE")) {
                // Format: UPDATE <id> <qtyChange>
                String[] parts = line.split("\\s+");
                int id = Integer.parseInt(parts[1]);
                int qtyChange = Integer.parseInt(parts[2]);

                Product p = inventory.get(id);
                if (p != null) {
                    p.quantity = p.quantity + qtyChange;

                    // Requirement: “Remove discontinued products”
                    // Since you said no delete/remove methods,
                    // we mark them as discontinued by setting quantity to 0.
                    if (p.quantity <= 0) {
                        p.quantity = 0;
                    }
                }
            } else if (line.equals("PRINT_AVAILABLE")) {
                printAvailableProducts(inventory);
            } else if (line.equals("PRINT_CATEGORY")) {
                displayCategoryWise(inventory);
            } else if (line.equals("PRINT_TOTAL_VALUE")) {
                double total = calculateTotalValue(inventory);
                System.out.printf("Total Inventory Value: %.2f%n", total);
            } else {
                // Unknown command, ignore
            }

            commands++;
            if (commands > 1000) break;
        }

        sc.close();
```

```
    }

    // Display all available products (quantity > 0)
    static void printAvailableProducts(Map<Integer, Product> inventory) {
        System.out.println("Available Products:");
        for (Product p : inventory.values()) {
            if (p.quantity > 0) {
                System.out.println(p.id + " " + p.name + " " + p.category + " " + p.quantity + " " + p.price);
            }
        }
    }

    // Display products category-wise
    static void displayCategoryWise(Map<Integer, Product> inventory) {
        Map<String, Integer> countByCategory = new HashMap<>();

        for (Product p : inventory.values()) {
            if (p.quantity > 0) {
                countByCategory.put(p.category, countByCategory.getOrDefault(p.category, 0) + 1);
            }
        }

        System.out.println("Category-wise (Available Products Count):");
        for (String cat : countByCategory.keySet()) {
            System.out.println(cat + " -> " + countByCategory.get(cat));
        }
    }

    // Calculate total inventory value = sum(quantity * price) for available products
    static double calculateTotalValue(Map<Integer, Product> inventory) {
        double total = 0.0;
        for (Product p : inventory.values()) {
            if (p.quantity > 0) {
                total += p.quantity * p.price;
            }
        }
        return total;
    }
}

// Product class
class Product {
    int id;
    String name;
    String category;
    int quantity;
    double price;

    Product(int id, String name, String category, int quantity, double price) {
        this.id = id;
        this.name = name;
        this.category = category;
        this.quantity = quantity;
        this.price = price;
    }
}
```

**INPUT**

```
5
101 Apple Electronics 50 2.5
102 Milk Groceries 30 1.2
103 Laptop Electronics 20 45.0
104 Bread Groceries 100 0.8
105 Soap Toiletries 40 1.5

UPDATE 102 10
UPDATE 105 -5
```

```
PRINT_AVAILABLE
PRINT_CATEGORY
PRINT_TOTAL_VALUE
```

**OUTPUT**

```
(no output)
```